

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

MAJOR PROJECT PROPOSAL**AXION****Adaptive eXchange for Identity via Oblivious Network***A Breach-Resistant, Risk-Adaptive Authentication SDK Built on the OPAQUE Protocol*

Submitted by

Mohammed Saad Shareef - 160923733202

Mohammad Saad Ur Rahman - 160923733205

Syed Ahsan Ahmed - 160923733211

Abdul Rahman Siddiqui Mohammed - 160923733216

4th Year, 7th Semester - B.E. Computer Science and Engineering
Academic Year 2026-27**1. Introduction & Problem Statement**

AXION stands for Adaptive eXchange for Identity via Oblivious Network - a name that reflects what the system actually does: it exchanges proof of identity using "oblivious" cryptographic techniques (explained below) that never expose a user's real password, and adapts its trust decisions in real time based on risk.

Almost every app today authenticates users the same basic way: a password is typed in, sent to a server, and checked against a stored (usually hashed) copy. If that server's database is ever breached - which happens to real companies constantly - attackers get a copy of every password hash and can attempt to crack them offline, often successfully, since most people reuse or choose weak passwords.

A cryptographic idea called Zero-Knowledge Proofs (ZKPs) promises a way around this: proving you know a password without ever transmitting or storing anything derived directly from it. However, simple ZK schemes applied naively to ordinary passwords (such as basic Schnorr proofs) have a hidden flaw - because passwords are low-entropy (people choose common, guessable ones), an attacker who steals the stored public value can still run a fast offline dictionary attack against it. A more advanced, purpose-built protocol called OPAQUE (currently being standardized by the IETF, the body that sets core Internet protocol standards) solves this specific weakness properly - but very few real applications have adopted it yet, because usable, developer-friendly implementations are still rare.

AXION closes this gap: a working, benchmarked, drop-in authentication SDK built on OPAQUE, combined with a real-time

AI risk-scoring layer that adds a second line of defence even if login credentials are technically valid but the login itself looks suspicious.

2. Objectives

- Implement the OPAQUE authenticated key-exchange protocol correctly, using audited cryptographic libraries rather than hand-built primitives.
- Package the protocol as a simple, pluggable SDK that any developer can integrate into a login flow with minimal code changes.
- Build an AI-based risk-scoring layer that evaluates device fingerprint, login velocity, and behavioural signals alongside every login attempt.
- Benchmark AXION's performance and security properties against traditional password-over-TLS authentication, producing real, measurable comparison data.
- Produce a demo application (Flutter) showing the entire flow working end-to-end, and a written evaluation suitable for submission as an IEEE conference paper.

3. Proposed System Overview

In simple terms: instead of a user's password ever being sent to or stored by a server, both sides run a short mathematical "conversation" that proves the user knows the correct password - without either side ever revealing or reconstructing it. Even if the server's entire database is stolen, there is nothing in it usable to recover a real password.

Architecture components:

- Client-side AXION SDK (Flutter/JavaScript) - handles the cryptographic exchange locally on the user's device.
- AXION Auth Server (Node.js) - verifies the OPAQUE exchange and never receives the raw password at any point, even during signup.
- Risk-Scoring Engine - a lightweight ML model (e.g., Isolation Forest or a small neural network) analysing device/behavioural signals per login attempt.
- Demo Application (Flutter) - a working login/signup flow demonstrating the entire system for evaluation and showcase purposes.

Workflow: User enters password → SDK runs the OPAQUE exchange with the server → server confirms proof of knowledge without ever seeing the password → risk-scoring engine evaluates the login context → session is granted, challenged with a second factor, or blocked based on combined cryptographic and behavioural trust.

4. Novel Contributions

To be precise and academically honest: the underlying idea of combining risk-based authentication with zero-knowledge techniques already exists in research literature, and adaptive/risk-based MFA is already common commercially (e.g., Okta, LoginRadius). AXION's real, defensible contribution is narrower and more concrete:

- A practical, open, developer-friendly SDK implementation of OPAQUE - a protocol that is standardized and provably secure, but still has very few accessible, ready-to-integrate libraries for common app stacks.
- Direct empirical benchmarking of OPAQUE-based authentication versus conventional password-over-TLS, on real metrics: login latency, computational overhead, and resistance to simulated offline dictionary attacks.
- Integration of this protocol with a lightweight, real-time behavioural risk-scoring layer in a single deployable package - a combination not currently available as an accessible open implementation.

This framing - a practical implementation and evaluation contribution, not a new cryptographic invention - is both honest and a legitimate, respected category of computer science project and publication.

5. Who Would Use AXION

- Fintech and healthtech companies needing strong, breach-resistant authentication to meet compliance requirements

(e.g., RBI/GDPR-style data protection expectations).

- SaaS platforms and app developers wanting a modern, passwordless-grade security upgrade without forcing users to change how they log in.
- Enterprises and educational institutions managing many user accounts, where a single database breach today would expose every password hash at once.
- Any development team currently relying on basic password-over-TLS authentication who want a drop-in upgrade rather than a full identity-system rebuild.

6. Business Model & Monetization

AXION is structured as developer infrastructure ("dev-tools"), similar in spirit to how Auth0, Okta, or Clerk operate - companies pay for the authentication layer; the end users logging into those companies' apps never pay anything directly.

Revenue Stream	How It Works
SDK Licensing (B2B SaaS)	Companies pay a monthly/annual fee to integrate the AXION SDK into their own apps, tiered by number of monthly active users - the standard dev-tools pricing model (same structure as Auth0/Okta).
Managed Auth-as-a-Service	Instead of self-hosting, smaller companies can use a fully managed AXION backend (we host the verification server), paying a per-authentication-call fee - similar to how Twilio charges per SMS/API call.
Enterprise Support & Compliance Add-ons	Larger clients (banks, hospitals) pay extra for dedicated support, audit logging, and compliance documentation needed for regulatory approval.
Open-Core Model	The core SDK can be open-source (driving adoption and credibility), while advanced features - the risk-scoring engine, analytics dashboard, enterprise support - remain paid, a proven model used by companies like GitLab and Sentry.

This is a genuinely investable structure: recurring B2B revenue, a real and growing market category (passwordless/breach-resistant authentication), and a natural upgrade path from free/open adoption to paid enterprise features.

7. Feasibility & Risk Mitigation

7.1 Cryptographic Correctness Risk

Implementing cryptography incorrectly is the single biggest risk in a project like this. AXION mitigates this by using existing, audited cryptographic libraries for the underlying primitives (elliptic curve operations, oblivious pseudorandom functions) rather than writing custom cryptographic code from scratch - the team's contribution is correct protocol integration and system design, not inventing new low-level cryptographic primitives.

7.2 Performance Overhead

OPAQUE requires more computation per login than a simple password check. This will be directly measured (not assumed) during the project, with login latency benchmarked on real devices, and optimizations applied if overhead is unacceptable for a mobile use case.

7.3 Adoption Friction for Real Companies

Any company adopting a new auth system must migrate existing user accounts. AXION's SDK will include a documented migration path (dual-support period where both old and new authentication work side by side) to make this a realistic, low-risk upgrade rather than an all-or-nothing switch.

7.4 AI Risk-Scoring False Positives

A risk engine that blocks legitimate users too aggressively would harm adoption. The model will be tuned and evaluated specifically for false-positive rate alongside detection accuracy, using simulated normal vs. anomalous login datasets, with

results reported transparently in the evaluation - not just accuracy claimed in isolation.

7.5 Technical Feasibility Within Timeline

Every component (OPAQUE reference implementations, Flutter, Node.js, standard ML libraries) is free, well-documented, and realistic for a four-person student team to build and evaluate properly within 6–8 months.

8. Research & Publication Plan

AXION is designed from the outset to produce a legitimate, IEEE-Xplore-indexed conference paper (a regional/applied-track venue, not a top-tier theoretical security conference - an honest and achievable target for an undergraduate team). The paper's contribution is implementation and empirical evaluation, and will include:

- A system architecture and threat-model description.
- Login latency and computational overhead benchmarks: AXION vs. traditional password-over-TLS.
- A simulated offline dictionary-attack comparison, demonstrating resistance where traditional systems fail.
- Risk-scoring model evaluation: precision, recall, and false-positive rate on simulated login data.
- A related-work section honestly citing OPAQUE, prior ZK-authentication research, and existing commercial risk-based auth systems - positioning AXION's contribution accurately as integration and evaluation.

This kind of applied, benchmarked systems paper is a realistic and valuable addition to a scholarship application or graduate school portfolio, precisely because it demonstrates real engineering execution rather than an unproven or overstated claim of invention.

9. Technology Stack

Layer	Tools
Client SDK	Flutter (mobile demo), JavaScript/TypeScript (web SDK)
Cryptographic Core	Audited OPAQUE/OPRF library (e.g., libsodium-based or existing OPAQUE reference implementation)
Auth Server	Node.js
Risk-Scoring Engine	Python (scikit-learn / lightweight neural network for anomaly detection)
Data Storage	PostgreSQL or Firebase Firestore (for account/session metadata only - never raw passwords)
Benchmarking & Evaluation	Custom load-testing scripts, Jupyter notebooks for result analysis and graphing

10. Team Roles & Contribution

Member	Responsibility
Abdul Rahman Siddiqui Mohammed	Cryptographic protocol integration (OPAQUE implementation), auth server logic, overall system architecture and benchmarking.
Mohammed Saad Shareef	Client SDK development (Flutter/JS), demo application, integration testing.
Mohammad Saad Ur Rahman	Risk-scoring engine - model design, training on simulated login data, false-positive tuning.
Syed Ahsan Ahmed	Evaluation & benchmarking design, performance testing, documentation, and paper drafting.

All four members contribute across both implementation and the research paper, ensuring balanced, demonstrable

individual contribution.

11. Expected Outcomes

A working, demoable authentication SDK implementing OPAQUE with an integrated AI risk-scoring layer, benchmarked against traditional authentication with real performance and security metrics, packaged as a genuine developer tool with a credible B2B monetization path - suitable for live demonstration, NBA accreditation showcase evidence, and submission as an IEEE-indexed applied research paper.

12. Tentative Timeline (6–8 Months)

Phase	Focus
Months 1–2	Study OPAQUE specification; select and integrate audited cryptographic libraries; basic client-server exchange working.
Months 2–4	Build the Flutter/JS SDK wrapper; implement the auth server; end-to-end signup/login flow functional.
Months 3–5	Design and train the risk-scoring engine on simulated login data; integrate with the auth flow.
Months 5–6	Full benchmarking: latency, overhead, simulated attack resistance, false-positive rate; begin paper drafting.
Months 6–7	Polish SDK, documentation, and demo app; refine paper based on results.
Months 7–8 (buffer)	Final report, pitch deck, demo rehearsal, paper submission preparation.

13. Conclusion

AXION takes a real, widely acknowledged weakness in how almost every application authenticates users today, and addresses it using a standardized, provably secure protocol that remains underused simply because accessible implementations don't yet exist. By building, benchmarking, and packaging that protocol properly - and pairing it with an adaptive AI risk layer - the team produces a technically serious, honestly-scoped, and genuinely useful piece of security infrastructure: strong enough for an NBA showcase, credible enough for an IEEE paper, and structured with a real path to becoming an actual developer tool business.